Tema 4

Definición de esquemas y vocabularios en XML

Lenguaje de marcas y sistemas de gestión de la información

Tech Hub <FP>
euroformac

ÍNDICE

Objetivos de la unidad 3
1.Introducción 3
2.Utilización de métodos de definición de documentos XML 4
2.1 ¿Qué es un Schema XML? 6
3.Creación de descripciones 8
3.1 Elementos 8
3.2 Elementos – Indicadores de ocurrencias 9
3.3 Elementos – Elementos complejos 9
3.4 Tipos de datos 13
3.4.1 Creación de tipos de datos simples personalizados 13
3.5 Atributos 14
3.6 Restricciones o facetas 15
4.Asociación con documentos XML 19
5.Validación 21
6.Herramientas de creación y validación 22
7.Documentación de especificaciones 23
8.Resumen 24
Glosario 25
Bibliografía 26

Objetivos de la unidad

- Establecer una estructura clara y coherente para la organización y presentación de datos en un documento XML.
- Definir reglas y restricciones sobre los tipos de datos que se pueden incluir en un documento XML, para garantizar que los datos sean precisos y coherentes.
- Facilitar la interoperabilidad entre diferentes sistemas y aplicaciones al definir un conjunto común de términos y vocabulario que se pueden utilizar para describir datos.
- Mejorar la eficiencia y la productividad al permitir la reutilización de esquemas y vocabularios definidos previamente.
- Proporcionar una base sólida para el procesamiento automatizado de documentos XML al establecer reglas claras para la validación y transformación de datos.

1. Introducción

La **validación de archivos XML** es un paso esencial en la gestión y procesamiento de documentos XML. Esta validación se realiza para garantizar que los archivos XML se hayan escrito correctamente y cumplan con las reglas sintácticas establecidas por los lenguajes de definición de esquemas, como DTD o XSD. La validación de archivos XML garantiza que el archivo tenga la estructura correcta, es decir, que contenga todas las etiquetas de apertura y cierre necesarias, que los atributos estén definidos correctamente y que se sigan las reglas de anidamiento adecuadas. La validación también puede incluir la verificación de la semántica del archivo XML, es decir, que los datos contenidos en el archivo tengan sentido y sean coherentes con las reglas de negocio y la lógica de la aplicación. La validación de archivos XML ayuda a garantizar la calidad de los datos, la integridad de la información y la interoperabilidad entre sistemas y aplicaciones que utilizan documentos XML.

2. Utilización de métodos de definición de documentos XML

Definición

Un **DTD** (Document Type Definition) es la descripción de la estructura y sintaxis de un documento XML o SGML.

Se usa para validar la estructura de múltiples documentos y mantener su consistencia.

En muchos aspectos los **DTD** se consideran anticuados y se usan cada vez menos, gracias, sobre todo, a la existencia de lenguajes de validación más modernos y más acomodados para validar los documentos XML.

Los **mayores problemas** de los DTD son:

- No indica cuál es el elemento raíz del documento (debe figurar en la etiqueta DOCTYPE).
- Ausencia de tipos de datos: no es posible indicarle, por ejemplo:
  - Que el contenido de la etiqueta <cantidad> sea un número positivo.
  - Que el contenido de la etiqueta <mes> sea un número entero entre 1 y 12.
  - Que el texto de la etiqueta <title> sea una cadena de hasta 255 caracteres.
  - No es un lenguaje XML. Esto conlleva la necesidad de tener distintos analizadores y API para el DTD y para el documento.
  - Son muy poco extensibles, y es difícil combinar diferentes DTD entre sí.
  - Apenas son compatibles con los espacios de nombres XML.
  - Acciones aparentemente sencillas se resuelven de manera complicada: por ejemplo, un elemento que contiene su vez tres elementos, pero el orden no tiene importancia.

El **XML Schema Definition Language** es una recomendación del Consorcio W3C que mejora en muchos aspectos a validación de documentos utilizando DTD.

Un **Schema XML** describe la estructura de un documento XML. El Schema XML es una alternativa al uso del DTD.

Importante

Muchas veces el lenguaje empleado en el Schema XML se denomina XSD (XML Schema Definition).

Ejemplo:

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:element name="correo">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="destinatario"
type="xs:string"/>
            <xs:element name="remitente" type="xs:string"/>
            <xs:element name="cabeceira" type="xs:string"/>
            <xs:element name="corpo" type="xs:string"/>
        </xs:sequence>
    </xs:complexType>
</xs:element>
</xs:schema>
```

Código fuente documento XSD

2.1 ¿Qué es un Schema XML?

El **objetivo de un Schema XML** es definir la estructura base de un documento XML, tal y como lo hace un DTD.

Un **Schema XML**:

- Define los elementos y atributos que pueden aparecer en un documento.
- Define qué elementos son elementos hijo.
- Define la orden de los elementos hijo.
- Define el número de elementos hijo.
- Define cuándo un elemento está vacío o cuándo puede incluir texto.
- Define tipos de datos para elementos y atributos.
- Define valores por defecto o fijos para elementos y atributos.

Las razones por las que los **Schemas XML** están reemplazando a los DTD son:

- Se pueden ampliar con futuras adiciones.
- Son más potentes y tienen más riqueza que los DTD.
- Están escritos en XML.
- Soportan tipos de datos.
- Soportan espacios de nombres.

Sabías que

En mayo del año 2001 el uso Schema XML pasó a ser una recomendación por parte de la organización W3C.

Una de las **grandes ventajas de los Schemas XML** es el soporte de tipos de datos. Gracias a esto es mucho más sencillo:

- Describir el contenido permitido dentro de un documento.
- Validar la exactitud del documento.
- Trabajar con datos de una base de datos.

- Definir restricciones de datos.
- Definir patrones de datos.
- Convertir datos entre diferentes tipos.

Los Schemas usan **sintaxis XML** por lo tanto:

- No tenemos que aprender un nuevo lenguaje.
- Podemos emplear el editor XML para editar nuestros archivos de Schema.
- Podemos manipular el Schema que el XML DOM.
- Podemos transformar el Schema con XSLT.

Los Schemas XML garantizan la comunicación segura de datos:

- Cuando enviamos datos desde un emisor a un receptor es muy importante que ambas partes sepan cómo van a recibir y enviar esos datos.
- Empleando los Schemas XML, el emisor puede describir los datos de una manera que el receptor comprenda perfectamente. Por ejemplo. Una fecha tal como "09-06-1992", en algunos países podrá ser interpretada como el 9 de junio y en otros países el 6 de septiembre.
- Con todo, un elemento XML con un tipo de datos "date" como, por ejemplo:

```
<date type="date">1992-06-09</date>
```

- Asegura el entendimiento mutuo entre el emisor y el receptor, ya que el tipo "date" en XML requiere el formato "YYYY-MM-DD".

Más info

Para ampliar información sobre Schema XML visualiza el vídeo XML Schema: Introducción

3. Creación de descripciones

La estructura de todo XSD es similar a la de un XML, pero presentando el elemento raíz <xs:schema>. Dicho elemento contiene una serie de atributos:

- **Xmlns:alias**: Se especifica el espacio de nombres de donde provienen los elementos y tipos usados, cuyo valor tiene que ser **"http://www.w3.org/2001/XMLSchema".** El alias suele ser xs o xsd, aunque, mientras se use la misma en todo el documento no importa el nombre que se le asigne. Hacen referencia a las etiquetas del espacio de nombres.
- **ElementFormDefault**: A la hora de declarar los elementos hay que indicar si debe añadirse el espacio de nombres delante, cuyos posibles valores son " qualified" y " unqualified" .
- **AtributeFormDefault**: A la hora de declarar los atributos hay que indicar si debe añadirse el espacio de nombres delante, cuyos posibles valores son " qualified" y " unqualified" , que indica que el primero de ellos debe ser obligatorio. Por defecto, se usa la segunda opción.
- **TargetNamespace**: Se especifica el espacio de nombres de los elementos definidos.
- **Version**: Se especifica la versión del documento de esquema.

Aunque no es obligatorio, la primera línea puede ser la declaración del XML. En la segunda, se define el esquema, con la especificación de los distintos elementos.

```
<?xml version="1.0" encoding="utf-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
elementFormDefault="qualified">
    .....
    Elementos
    .....
</xs:schema>
```

Estructura de un documento XSD

3.1 Elementos

Puede realizarse una distinción entre los **tipos de elementos de un XSD**. Están los elementos **simples** y **complejos**. Elementos sin hijos ni atributos o simples

se definen con la marca **<xs:element>,** conteniendo solamente valores de un determinado tipo. Donde:

- **Nombre**: Se representa mediante el atributo name, asignándole el valor que se desee.
- **Tipo**: Se usa el atributo type, que permite especificar el tipo de elemento.
- **Valor**: Puede ser un valor por defecto (default) que permite ser modificado o un valor obligatorio (fixed) que no podrá cambiarse. Cada uno de ellos irá acompañado de su correspondiente asignación.

Ejemplo:

```
<xs:element name="color" type="xs:string" default-
t="azul"/>
```

3.2 Elementos – Indicadores de ocurrencias

Indica las veces que un elemento debe aparecer o repetirse mediante dos atributos que informan del número máximo y mínimo de veces que puede aparecer:

- **MaxOccurs**: Se especifica un valor con el número máximo de veces que puede aparecer el elemento, cuyo valor por defecto es 1. Para especificar un número de veces ilimitado, el valor es " unbounded" .
- **MinOccurs**: Se especifica un valor con el número mínimo de veces que puede aparecer el elemento, cuyo valor por defecto 1. Para especificar un número de veces ilimitado, el valor es " unbounded" .

3.3 Elementos – Elementos complejos

Contienen otros elementos y atributos. Dependiendo del contenido, pueden darse ejemplos de elementos con hijos, con contenido vacío y atributos, con contenido no vacío y atributos, entre otros.

Se definen con el tipo de datos **<xs:complexType>,** seguido de un indicador de orden y los elementos simples, que puede variar dependiendo del contenido que tenga.

Ejemplo:

```
<xs:element name="nombreElementoCompuesto">
   <xs:complexType>
      <Indicar_Orden>
         Elementos / Atributos
      </Indicar_Orden>
   </xs:complexType>
</xs:element>
```

Donde:

- <xs:complexType>: se usa para definir elementos de tipo complejo.
- Indicar_Orden: en este apartado hay que especificar un indicador de orden de aparición de los distintos elementos, que puede ser tres distintos:
  - <xs:sequence>: se especifican los hijos del elemento principal, siguiendo la secuencia u orden de aparición indicada.
  - <xs:choice>: se especifica el elemento hijo que puede aparecer de entre todos. Solo uno de ellos.
  - <xs:all>: los hijos pueden aparecer en cualquier orden de una sola vez, la secuencia aparecerá en el orden que se desee.

Ejemplo con declaración de un XSD con elementos complejos mediante referencias:

```
<xs:schema elementFormDefault="qualified”
xmlns:xs="http://www.w3.org/2001/XMLSchema"
<xs:element name="usuario">
   <xs:complexType>
   <xs:sequence>
      <xs:element ref="nombre"/>
      <xs:element ref="email"/>
      <xs:element ref="fechaNac"/>
   </xs:sequence>
   </xs:complexType>
</xs:element>
<xs:element name="nombre" type="xs:string" />
<xs:element name="email" type="xs:string" />
<xs:element name="fechaNac" type="xs:date" />
</xs:schema>
```

Ejemplo anterior con declaración de un XSD con elementos simples:

```
<xs:schema elementFormDefault="qualified"
xmlns:xs="http://www.w3.org/2001/XMLSchema"
<xs:element name="usuario">
   <xs:complexType>
   <xs:sequence>
      <xs:element name="nombre" type="xs:string" />
      <xs:element name="email" type="xs:string" />
      <xs:element name="fechaNac" type="xs:date" />
   </xs:sequence>
   </xs:complexType>
</xs:element>

</xs:schema>
```

Ejemplo: Un XML correcto a la hora de validar el ejemplo anterior.

```
<?xml version="1.0" enconding="iso-8859-1"?>
<usuario>
   <nombre>Pedro García </nombre>
   <email>usuario@gmail.com </email>
   <fechaNac>2020-10-15</fechaNac>
</usuario>
```

Si, por el contrario, el usuario se equivoca a la hora de construir el XML siguiendo el esquema, dará error al validar.

El indicador de orden <xs_choice> permite especificar uno de los elementos hijos que aparecen, pero solo uno de ellos.

Ejemplo: Se pretende que el usuario se identifique con su nombre de usuario o email, con uno de los dos.

```
<xs:schema elementFormDefault="qualified"
xmlns:xs="http://www.w3.org/2001/XMLSchema"
<xs:element name="usuario">
   <xs:complexType>
      <xs:choice>
         <xs:element name="nombre" type="xs:string" />
         <xs:element name="email" type="xs:string" />
      </xs:choice>
   </xs:complexType>
</xs:element>
</xs:schema>
```

Ejemplo: Un XML correcto a la hora de validar el ejemplo anterior.

```
<?xml version="1.0" enconding="iso-8859-1"?>
   <usuario>
      <email>usuario@gmail.com </email>
   </usuario>
```

El indicador de orden `<xs:all>`, permite especificar la secuencia de elementos en cualquier orden, aunque sin poder añadir dentro de él `<xs:choice>` o `<xs:sequence>`.

Ejemplo:

```
<xs:schema elementFormDefault="qualified"
xmlns:xs="http://www.w3.org/2001/XMLSchema"
   <xs:element name="usuario">
      <xs:complexType>
         <xs:all>
            <xs:element name="nombre" type="xs:string" />
            <xs:element name="email" type="xs:string" />
            <xs:element name="fechaNac" type="xs:xs:date"
/>
         </xs:all>
     </xs:complexType>
   </xs:element>
</xs:schema>
```

Ejemplo: Un XML correcto a la hora de validar el ejemplo anterior, podría ser:

```
<?xml version="1.0" enconding="iso-8859-1"?>
<usuario>
   <nombre>Pedro García </nombre>
   <fechaNac>2016-10-15</fechaNac>
   <email>usuario@gmail.com </email>
</usuario>
```

Más info

Para ampliar información sobre Schema XML: Estructura visualiza el vídeo [XML Schema: Estructura]

3.4 Tipos de datos

XML Schema dispone de numerosos tipos de **datos simples predefinidos** incorporados para los elementos o atributos, siendo los más comunes los que se observan a continuación:

- **Decimal**: Número decimal.
- **Boolean**: Un único valor lógico (1,0).
- **DateTime**: Fecha y hora en formato (AAA-MM-DD T HH:MM:SS).
- **String**: Texto o cadena de caracteres.
- **Date**: Fecha: año-mes-día (aaaa-mm-dd).
- **Time**: Hora: hh:mm:ss.
- **Integer**: Número entero positivo o negativo.
- **PositiveInteger**: Número entero positivo.
- **Long**: Entero de 64 bits.
- **Short**: Entero de 16 bits.

3.4.1 Creación de tipos de datos simples personalizados

A veces, es interesante crear tipos de datos nuevos personalizados, dotándolos de mayor precisión y eficiencia.

Estos pueden ser asignados a elementos o atributos, siendo reutilizados en el esquema XSD.

Para definir un tipo simple nuevo, se usa la siguiente estructura <xs:simpleType>, cuyo atributo name servirá para asignárselo al elemento e indicar que es de un tipo simple nuevo.

Dentro de un tipo simple, pueden añadirse restricciones o facetas, listas, uniones, etc.

La sintaxis puede apreciarse en el siguiente ejemplo:

```
<xs:simpleType name="Asignatura">
.....
</xs:simpleType>
```

Más info

Para ampliar información sobre Schema XML: Tipos de datos simples, visualiza el vídeo XML Schema: Tipos de datos simples

Para ampliar información sobre Schema XML: Tipos de datos complejos, visualiza el vídeo XML Schema: Tipos de datos complejos

3.5 Atributos

Los **atributos** poseen la misma funcionalidad que los atributos de un elemento DTD. Los elementos con atributos se consideran ele**mentos complejos**, declarándose como **tipos simples**. Intentan dotarlos de más información usando la siguiente sintaxis. Al igual que en los elementos, se pueden añadir restricciones a sus tipos.

Estructura:

```
<xs:attribute NombreA TipoA USO ValorA>
```

Donde:

- **NombreA**: Se representa mediante el atributo name, asignándole el valor que se desee.
- **TipoA**: Se usa el atributo type, que permite especificar el tipo de elemento.
- **USO**: Se especifica la opcionalidad o no del atributo mediante use, que puede tener los valores: “ prohibited” , que indica que es prohibido; “ required” para indicar que es obligatorio, y “ optional” , que informan de que es opcional y es el valor por defecto.
- **ValorA**: Puede ser un valor por defecto (default) o un valor fijo u obligatorio (fixed) con su correspondiente asignación.

Ejemplo:

```
<xs:attribute name="dni" type="xs:integer" use="required">
```

3.6 Restricciones o facetas

Si se necesita o se quiere poner una **restricción**, también llamada **facetas**, a un atributo o elemento, con un DTD no se podía especificar.

XML Schema permite especificar esta característica controlando los datos introducidos en el XML.

- **Length**: Contiene un atributo value donde se especifica un valor de longitud fija.
- **MinLength**: Contiene un atributo value donde se especifica el valor de longitud mínima.
- **MaxLength**: Contiene un atributo value donde se especifica el valor de longitud máxima.
- **WhiteSpace**: Contiene un atributo value donde se especifica un valor que puede ser (“ preserve” , “ replace” y “ collapse”). Trata del uso de los espacios en blanco, retornos de carro, etc., que puedan aparecer.
- **MaxExclusive:** Contiene un atributo value donde se especifica el valor, que es menor al especificado.
- **MinExclusive:** Contiene un atributo value donde se especifica el valor, que es mayor al especificado.
- **MaxInclusive:** Contiene un atributo value donde se especifica el valor, que es mayor o igual al especificado.
- **MinInclusive:** Contiene un atributo value donde se especifica el valor, que es menor o igual al especificado.
- **TotalDigits:** Contiene un atributo value donde se especifica el valor, que es el número máximo de dígitos de un numero teniendo en cuenta los decimales.
- **FractionDigits:** Contiene un atributo value donde se especifica el valor, que es el número máximo de dígitos de decimales de un número.
- **Enumeration:** Contiene un atributo value donde se especifica uno de los valores admitidos de la lista.
- **Pattern:** Contiene un atributo value donde se especifica un rango de caracteres admitidos o expresión regular.

Ejemplo: El siguiente documento XML almacena datos relativos a varios productos, servirá para explicar las restricciones.

```
<?xml version="1.0" enconding="iso-8859-1"?>
<bdproductos xmlns:xsi:"http://www.w3.org/2001/XMLSchema-
instance" xsi:no-NamespaceSchemaLocation="zax.xsd">
   <producto>
      <nombre>teclado</nombre>
      <codigo>A001</codigo>
      <categoria>A</categoria>
      <iva>21</iva>
      <precio>15</precio>
      <descuento>10</descuento>
   </producto>

   <producto>
      <nombre>monitor</nombre>
      <codigo>A002</codigo>
      <categoria>B</categoria>
      <iva>21</iva>
      <precio>155.5</precio>
      <descuento>10</descuento>
    </producto>
</bdproductos>
```

Ejemplo: El esquema asociado donde algunos de los tipos de los elementos son tipos simples nuevos personalizados, con una serie de restricciones cada uno de ellos.

```
<?xml version="1.0" enconding="iso-8859-1"?>
<xs:schema elementFormDefault="qualified"
xmlns:xs="http://www.w3.org/2001/XMLSchema"
   <xs:element name="bdproductos">
      <xs:complexType>
         <xs:secuence>
            <xs:element maxOccurs="unbounded"
ref="producto"/>
         </xs:secuence>
      </xs:complexType>
   </xs:element>
   <xs:element name="producto">
      <xs:complexType>
         <xs:secuence>
            <xs:attribute name="nombre" type="xs:string"/>
            <xs:attribute name="codigo"
type="xs:Tlongitud"/>
            <xs:attribute name="categoria"
type="xs:Tcategoria"/>
            <xs:attribute name="iva" type="xs:integer"/>
```

```
            <xs:attribute name="precio" type="xs:Tprecio"/>
            <xs:attribute name="descuento"
type="xs:Tdescuento"/>
        </xs:secuence>
     </xs:complexType>
   </xs:element>
</xs:schema>
```

Para añadir una restricción se usa el elemento **xs:restriction**, que define las restricciones del tipo. Dispone de un atributo base, que indica el tipo de datos.

Ejemplo: Descuento puede tener el valor entre 1 y 10, inclusive.

```
<xs:simpleType name="Tdescuento">
   <xs:restriction base="xs:integer">
      <xs:minInclusive value="1" />
      <xs:maxInclusive value="10" />
   </xs:restriction>
</xs:simpleType>
```

En las restricciones también se pueden presentar el uso de patrones, donde el valor permitido es uno de los del rango de valores permitidos. Algunas de las expresiones regulares o patrones que se pueden incluir son:

- **[0-9]:** Número del 0 al 9.
- **[a-z]:** Letra minúscula perteneciente a ese intervalo.
- **[A-Z]:** Letra mayúscula perteneciente a ese intervalo.
- **[A-Z a-z]:** Letra tanto en mayúscula como en minúscula perteneciente a ese intervalo.
- **[aeiou]:** Un carácter de los que aparecen entre los corchetes.
- **[^aeiou]:** Un carácter que no aparezca entre corchetes.
- **{X}:** Las llaves indican que tiene que repetirse X veces el contenido que haya delante.
- **(A):** Cadena que coincide con A.
- **A|B:** Cadena que es igual a la cadena A o a la cadena B.
- **AB:** Cadena que es la concatenación de las cadenas A y B.
- **A?:** Cero o una vez la cadena A.
- **A+:** Una o más veces la cadena A.

- **A*:** Cero o más veces la cadena A.

Ejemplo: En el campo categoría solo puede contener caracteres en mayúsculas A o B o C o D o E.

```
<xs:simpleType name="Tcategoria">
   <xs:restriction base="xs:string">
      <xs:pattern value="[A-E]"/>
   </xs:restriction>
</xs:simpleType>
```

Todas están restricciones se añadirán **al final** del archivo .xsd **antes** del cierre de la etiqueta **</xs:schema>**

Más info

Para ampliar información sobre Schema XML: Diseño, visualiza el vídeo XML Schema: Diseño

4. Asociación con documentos XML

La **asociación de un documento XSD** a uno XML se realiza mediante un espacio de nombres con una serie de atributos:

- **Xmlns**: Definir el espacio de nombre indicando que van a usarse los elementos definidos en la URL especificada. Los elementos del esquema llevaran el prefijo que se defina, en este caso será xs:

```
xmlns:xs="http://www.w3.org/2001/XMLSchema-instance"
```

La referencia a los esquemas puede realizarse mediante los siguientes atributos:

- **NoNamespaceSchemaLocation:** Se emplea cuando se utilizarán espacios de nombres en el documento. Se usará un fichero con el esquema:

```
xs:noNamespaceSchemaLocation = "archivo.xsd"
```

- **SchemaLocation**: Se emplea cuando se utilizan explícitamente los nombres de los espacios de nombres en las etiquetas.

Ejemplo: El documento XML de bases de datos de alumnos hace referencia a un fichero de espacio de nombres previamente creado, que servirá para que sea validado el XML.

```
<?xml version="1.0" enconding="iso-8859-1"?>
<bds xmlns:xsi:"http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="xsalumnos.xsd">
   <alumno>
      <nombre>María</nombre>
      <padre>Ramón</padre>
      <madre>Sara</madre>
      <edad>5</edad>
      <ciudad>Madrid</ciudad>
   </alumno>
   <alumno>
      <nombre>Carlos</nombre>
      <padre>Pedro</padre>
      <madre>Isabel</madre>
      <edad>5</edad>
      <ciudad>Madrid</ciudad>
   </alumno>
</bdalumnos>
```

Documento Schema XML.

```
<?xml version="1.0" enconding="iso-8859-1"?>
<xs:schema attributeFormDefault= "unqualified"
elementFormDefault="qualified"
xmlns:xs="http://www.w3.org/2001/XMLSchema">
   <xs:element name="bdalumnos">
      <xs:complexType>
         <xs:secuence>
            <xs:element maxOccurs="unbounded"
name="alumno">
               <xs:complexType>
                  <xs:secuence>
                     <xs:attribute name="nombre"
type="xs:string"/>
                     <xs:attribute name="padre"
type="xs:xstring"/>
                     <xs:attribute name="madre"
type="xs:xstring"/>
                     <xs:attribute name="ciudad"
type="xs:xstring"/>
                  </xs:secuence>
               </xs:complexType>
            </xs:element>
         </xs:secuence>
      </xs:complexType>
   </xs:element>
</xs:schema>
```

5. Validación

El **método de validación** es un proceso que permite comprobar que el contenido de un fichero XML presenta una estructura adecuada y cumple una serie de normas (tipo de elementos y atributos que pueden añadirse, orden de aparición, etc.)

Un documento válido tiene que estar bien formado y cumplir una serie de requisitos especificados en una estructura XML Schema.

Las herramientas que se encargan de realizar se denominan **validadores** o **parsers**.

Sabías que

Un documento puede estar bien formado, pero no ser válido. En cambio, si es válido, seguro que está bien formado.

Existen numerosas aplicaciones de escritorio del ámbito de XML que ofrecen la opción de comprobar si un documento está bien formado y es válido.

También podemos encontrar aplicaciones online como:

- Free Online XML Validator (XSD)
- XML Validator – XSD (XML Schema)

6. Herramientas de creación y validación

Existen multitud de herramientas relacionadas con el manejo de XML y sus tecnologías asociadas (DTD, XSD, XPATH, XSLT, …). Algunas son muy sofisticadas y, en general, con licencias propietarias. Hay otras más sencillas también de código abierto.

Entre las ya vistas en la unidad anterior también tenemos el Visual Studio Code.

Más info

Para conocer cómo instalar y configurar el Visual Studio Code, así como para validar documentos XML puedes visualizar el vídeo Usar VSCode para validación

7. Documentación de especificaciones

La creación de un esquema puede ser algo complejo, por lo que introducir datos relacionados con la autoría, utilidad, derechos de autor, etc., es un aspecto que ha de tenerse en cuenta.

Si se necesita añadir una serie de información a los esquemas para que pueda ser consultada por otros usuarios, se usan las siguientes instrucciones:

- **Xs:annotation**: Especifica los comentarios dentro de un esquema que actúan como documentación de este. Tiene un atributo opcional (id) y dos elementos hijos (appinfo o documentation).
- **Xs:appinfo**: Se usa para especificar la información de la aplicación. Tiene un atributo opcional (source).
- **Xs:documentation**: Añade comentarios en un esquema. Debe ir dentro de xs:annotation y tiene dos atributos opcionales (source y xml:lang).

Ejemplo:

```
<xs:schema elementFormDefault="qualified"
xmlns:xs="http://www.w3.org/2001/XMLSchema">
<xs:annotation>
   <xs:appinfo>Lenguajes de marcas</xs:appinfo>
   <xs:documentation xml:lang="es"> comentarios sobre
   </xs:documentation>
</xs:annotation>
```

8. Resumen

XML es un lenguaje de marcado que permite estructurar la información de manera jerárquica mediante etiquetas que representan los datos y su relación entre ellos. XML Schema, por su parte, es una herramienta utilizada para crear definiciones de documentos XML y establecer reglas que determinan la estructura y el contenido permitido en los documentos.

Para asegurarse de que un documento XML sea correcto y válido, se deben seguir ciertas reglas sintácticas establecidas en los DTD o XSD. La validación de un documento XML es importante para garantizar que cumple con las especificaciones y normas establecidas, evitando errores y fallos en el procesamiento de la información.

Los desarrolladores y programadores cuentan con diferentes herramientas de edición y validación para asegurar que los documentos XML cumplan con las normas y especificaciones establecidas. Un documento XML puede estar bien formado, es decir, seguir las reglas sintácticas establecidas, pero no ser válido si no cumple con las reglas semánticas establecidas en el DTD o XSD. En cambio, si un documento XML es válido, se puede estar seguro de que cumple con las reglas sintácticas y semánticas establecidas en el DTD o XSD.

Glosario

- **JSON**: Sigla de JavaScript object notation. Formato de texto ligero que se usa para el intercambio de datos de facil interpretación.
- **SGML**: Signal de standard generalized markup language. Estándar para lenguajes de marcado.
- **Xpath**: Sigla de XML Path Language. Lenguaje usado para procesar documentos XML.
- **XSLT**: XSLT o XSL Transformaciones es un estándar de la organización W3C que presenta una forma de transformar documentos XML en otros, incluso a formatos que no son XML. La transformación de un documento mediante hojas de estilo XSLT se lleva a cabo mediante reglas de plantilla que se aplican al documento original que se quiere transformar. Dichas reglas de plantilla están vinculadas al documento fuente y pueden ser una o varias para lograr la transformación deseada.
- **XML**: Lenguaje de marcado propio y con una sintaxis relativamente sencilla.
- **XML Schema**: Herramienta utilizada para crear definiciones de documentos, analizando su sintaxis y estructura.
- **Vocabulario XML**: Conjunto de elementos y atributos que se utilizan para describir un tipo específico de documento XML.
- **Elementos**: Componentes básicos de un documento XML, como etiquetas, atributos y valores.
- **Indicadores de ocurrencias**: Elementos que permiten especificar cuántas veces puede aparecer un elemento en el documento.
- **Elementos complejos**: Elementos que contienen otros elementos o atributos dentro de ellos.
- **Tipos de datos**: Categorías predefinidas que se utilizan para especificar el tipo de información que se almacena en un elemento o atributo.

Bibliografía

- Zurdo, S. F. J. T. R. & RA-MA Editorial. (2023). LENGUAJES DE MARCAS Y SISTEMAS DE GESTIÓN DE LA INFORMACIÓN. CFGS. (1.a ed.). RA-MA Editorial.
- Castro, J. M., & Rodríguez, J. R. (2001). Lenguajes de marcas y sistemas de gestión de la información. Garceta.
- XML Schema Tutorial. (s. f.). https://www.w3schools.com/xml/schema_intro.asp
- XSD (XML Schema Definition) Tutorial. W3schools. https://www.w3schools.blog/xsd-xml-schema-definition-tutorial